Exercise 20: A2D_Intf & inertial_integrator

- There are 2 parts to this exercise
- You will split your team into 2 teams of 2
  - One team works on **A2D_Intf**
  - One team works on **inertial_integrator**
- Once your part is completed, you should fill the other half of the team in on how it was done
  - Everyone in the team should have basic knowledge of every component in the final project
- Make **one** Gradescope submission, including all team members in it (instructions)

A2D Intf Design

- The DE0-Nano has on-board A2D converter (ADC128S)
  - SPI based device
- We will use 4 channels of the A2D converter for our Segway
  - Channel 0 → Left load cell
  - Channel 4 → Right load cell
  - Channel 5 → Steering Potentiometer
  - Channel 6 → Battery Voltage
- We will use our **SPI_main** to access it
  - We will perform two transactions nearly back to back
  - First transaction we tell the ADCC128S what channel to convert
  - Second transaction we read the converted analog data for that channel

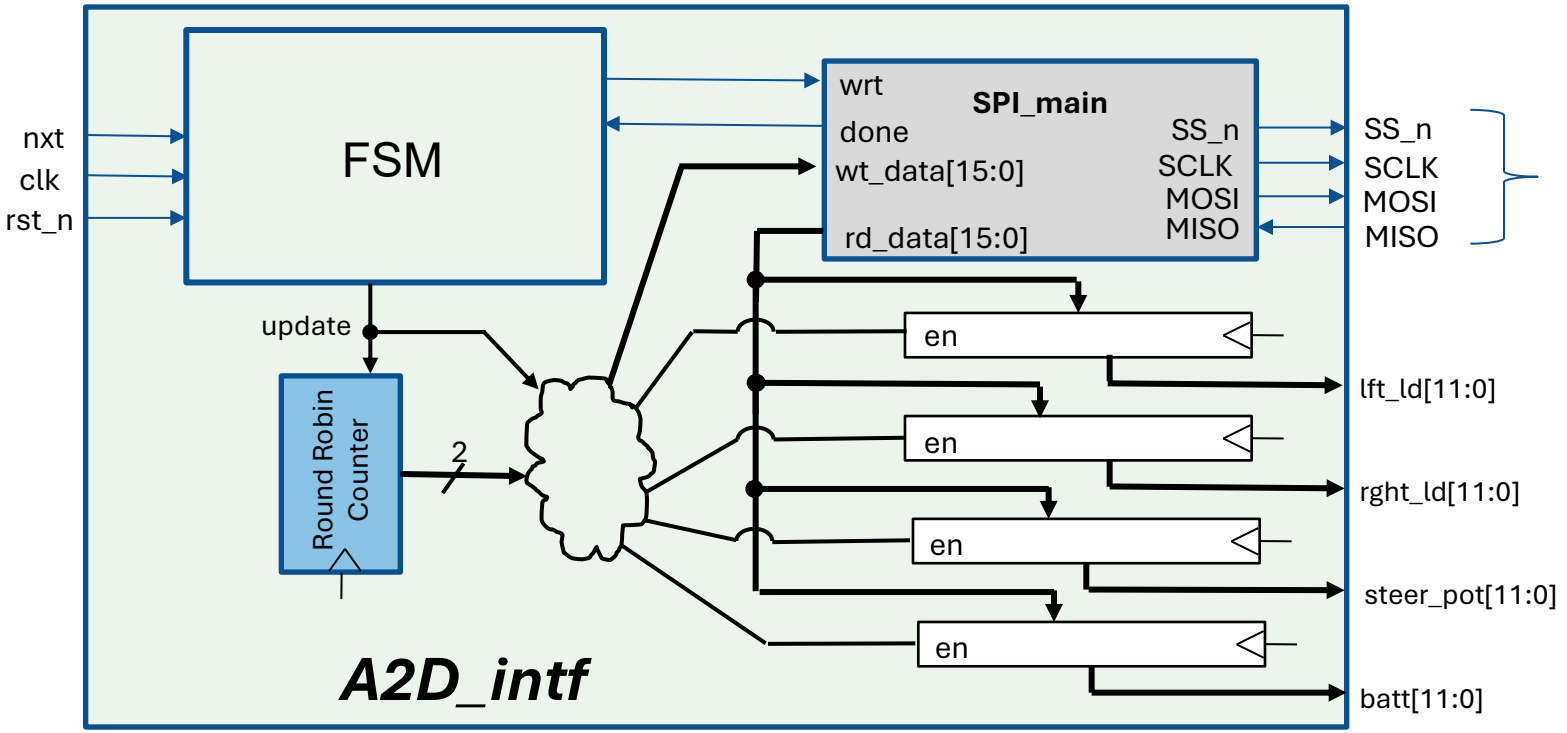
A2D Intf Design

- Since there are 4 channels we want to read, we will read them in a **round robin** fashion

- You will need some **holding registers** for the values you've read
- You will have a **round robin counter** to determine which channel to read and which holding register to update
- You will need an **FSM** to control when to initiate SPI transactions
  - The SM will sit idle till it is told to perform a conversion (**nxt** asserted)
  - Then it will kick off **two** SPI transactions via SPI_main.
  - The first SPI transaction determines what channel to convert, and the second SPI transaction reads the result for that channel.
  - The round robin counter is then incremented, and on the **nxt** request it will convert the next channel in the sequence.

- This module should be called **A2D_Intf.sv**

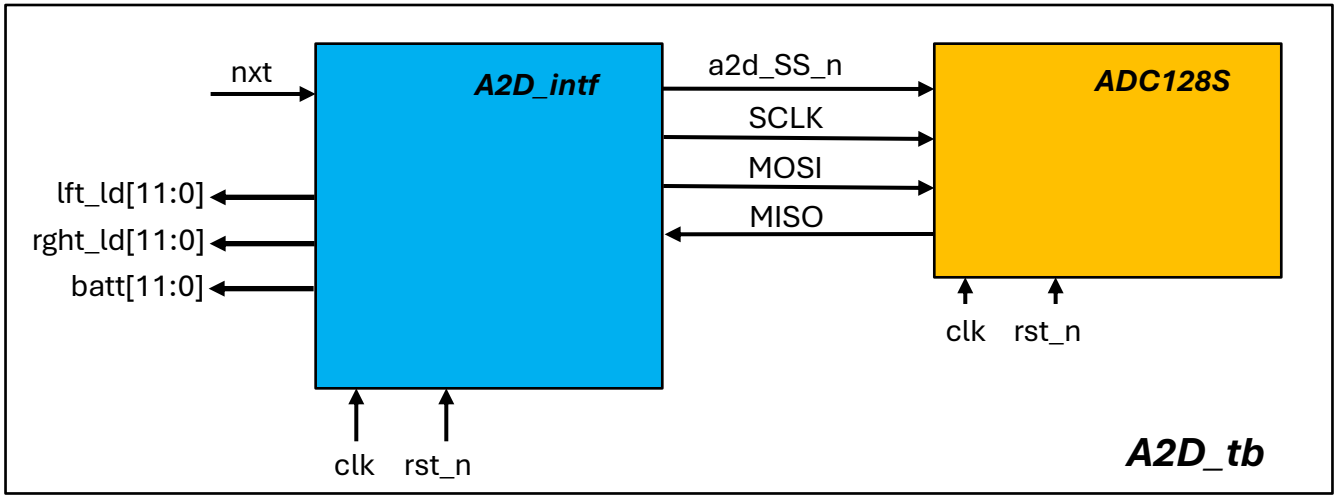
A2D Intf Design



Signal	Dir	Description
clk,rst_n	in	50MHz clock and active low asynch reset
nxt	in	Initiates A2D conversion on next measurand
lft_ld[11:0]	out	Result of last conversion on channel 0 (left load cell)
rght_ld[11:0]	out	Result of last conversion on channel 4 (right load cell)
steer_pot[11:0]	out	Result of last conversion on channel 5 (steering potentiometer)
batt[11:0]	out	Result of last conversion on channel 5 (battery voltage)
SPI Interface	Out/in	SS_n, SCLK, MOSI, MISO of a SPI interface. Comes from copy of SPI_main embedded into this unit.

A2D Intf Testbench

- We provide a ***model*** of the on-board A2D converter called **ADC128S.sv**
- Create the following testbench and make it self-checking



- **ADC182S** will return the following values:

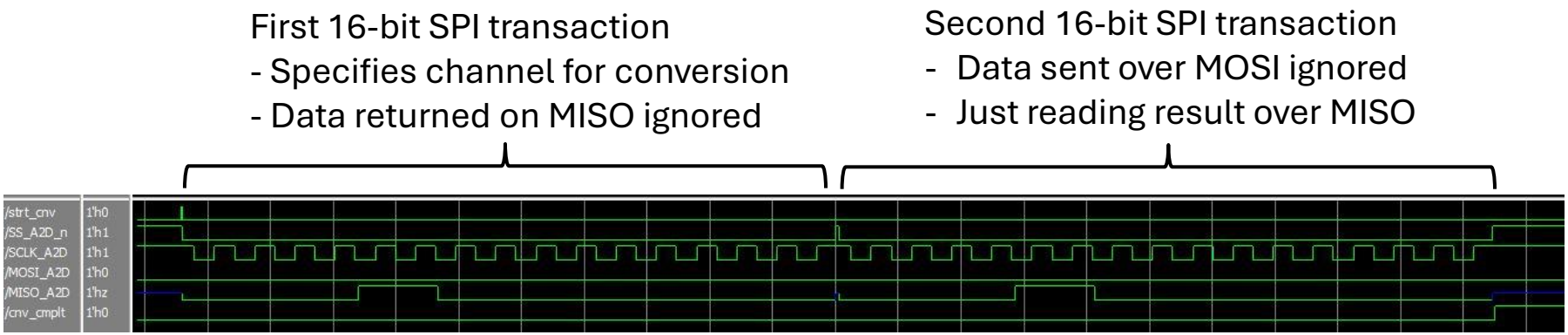
channel	6	5	4	0
1st read	0xC06	0xC05	0xC04	0xC00
2nd read	0xBF6	0xBF5	0xBF4	0xBF0
3rd read	0xBE6	0xBE5	0xBE4	0xBE0
…	…	…	…	…

0x010 less for every subsequent reading

- If 1st channel read was 0 and 2nd read was 4, then you get: 0xC00, 0xBF4

A2D Intf Testbench

- Your waveform should look something like this:

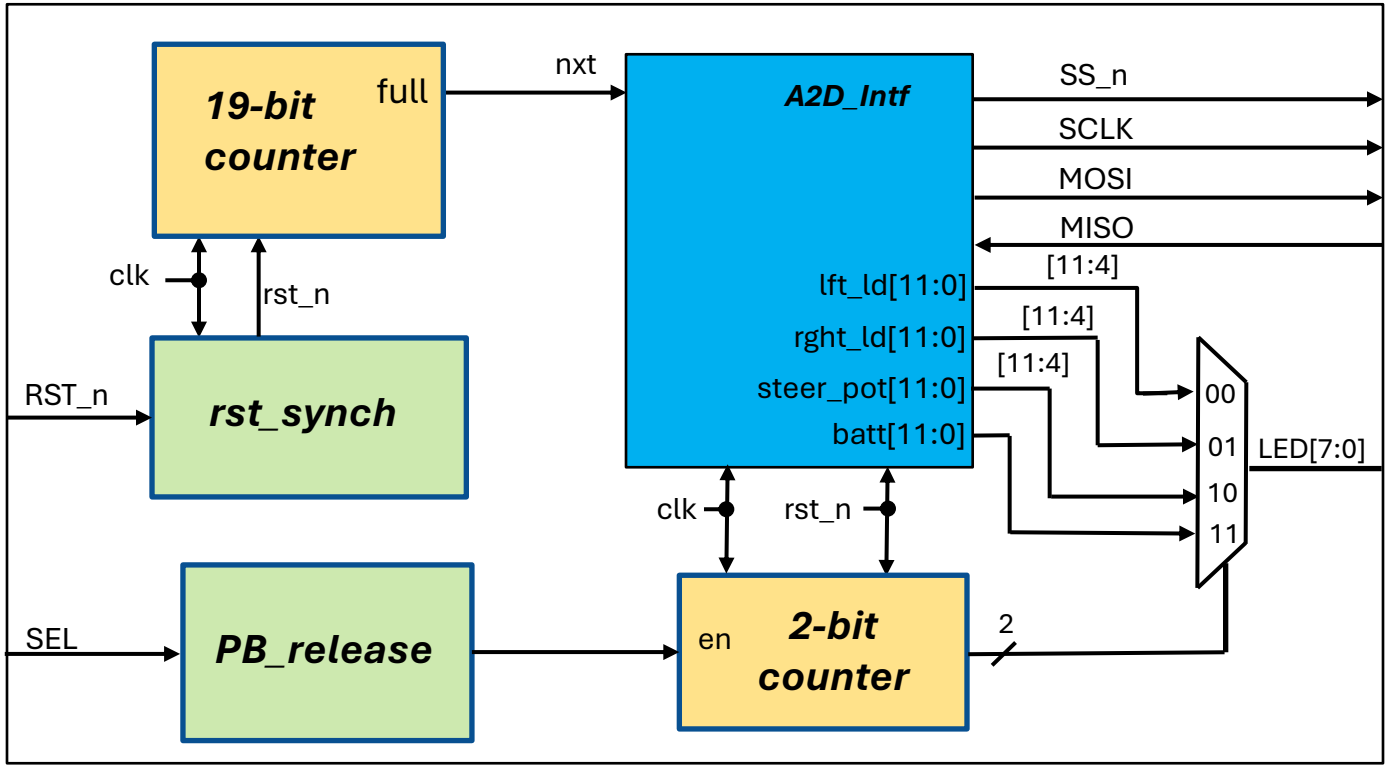


- You have two 16-bit SPI transactions separated by 1 system clock cycle
- The command to request a conversion is `{2'b00,channel[2:0],11'h00}`
  - The upper 2 bits are always 0
  - The next 3 bits specify the channel to convert
  - The lower 11 bits are always 0

- 1st transaction: we send 0x0000 to A2D over MOSI (e.g., channel 0)
- 2nd transaction: we read 16'h0C00 on MISO (e.g., 12'hC00)

Mapping A2D_Intf to DE0 Nano

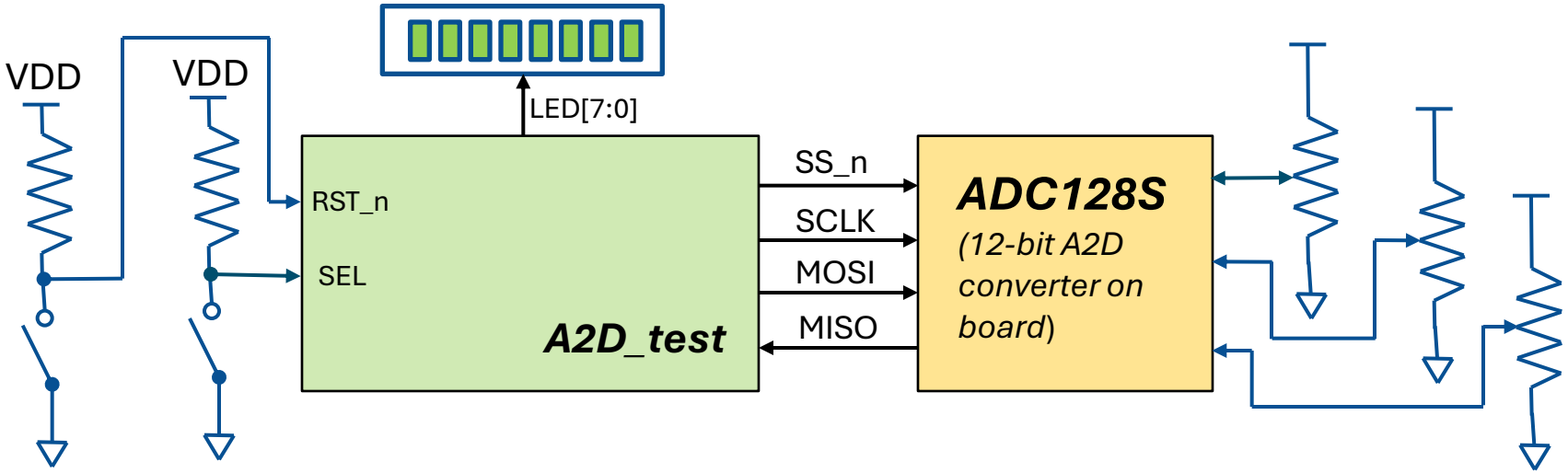
- A shell for **A2D_test.sv** is provided. Download and complete it.



- Make a 19-bit free running counter with a **full** output. This will drive **nxt** so we get roughly 100 conversions a second.
- The **PB_release** block will drive a 2-bit counter that selects which output we are looking at

Mapping A2D_Intf to DE0 Nano

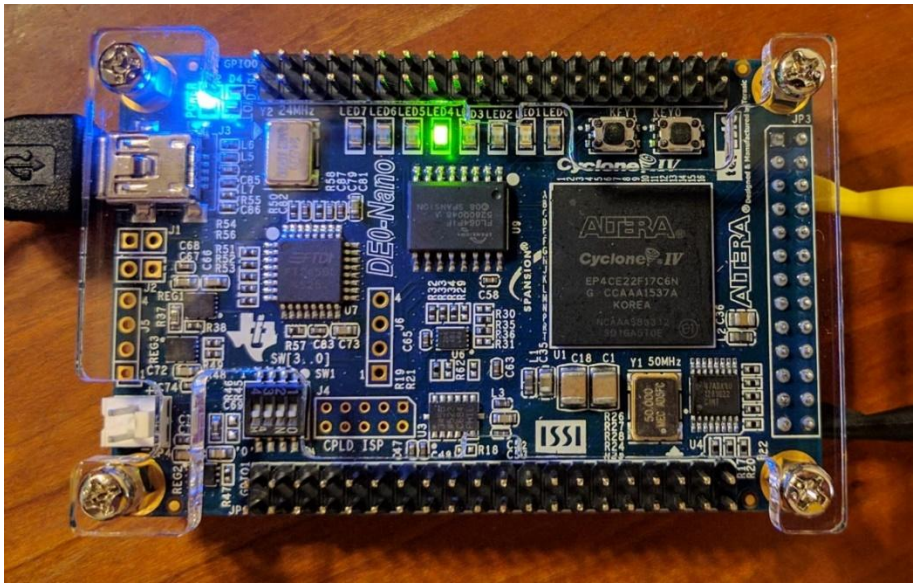
- Your **A2D_test.sv** will use *push buttons* for **RST_n** and **SEL**, and will display the converted result on the *LEDs*

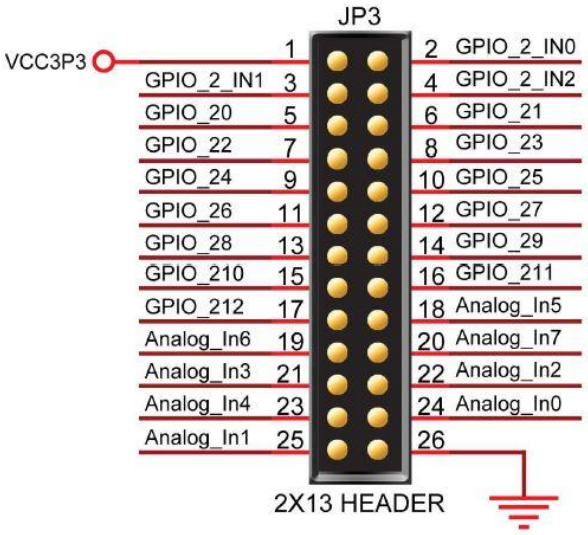


Signal	Dir	Description
clk	in	50MHz clock
RST_n	in	Unsynchronized input from push button
SEL	in	From 2nd push button. Selects next thing to look at (lft,rght,batt)
LED[7:0]	out	Upper 8-bits of conversion results of selected output
SPI Intf	out/ in	The SS_n, SCLK, MOSI, and MISO of SPI interface to the A2D converter on DE0-Nano

Mapping A2D_Intf to DE0 Nano

- There are Quartus project file and settings file available for download:
  - **A2D_test.qpf**, **A2D_test.qsf**
- Ensure that all necessary files are added to your project
- Compile and ensure no build errors
- Call anyone on the teaching staff over to Demo
  - Bonus credit: +1% if your group demos a correct implementation by Friday





Inertial Integrator

- The pitch readings we obtain from the gyro are ***angular rates*** (°/sec). We need to integrate to get degrees of rotation.
- Integration simply means summing over time.
  - We need an accumulator register that sums in the signed angular rate readings we get:

    ptch_int = ptch_int – ptch_rt

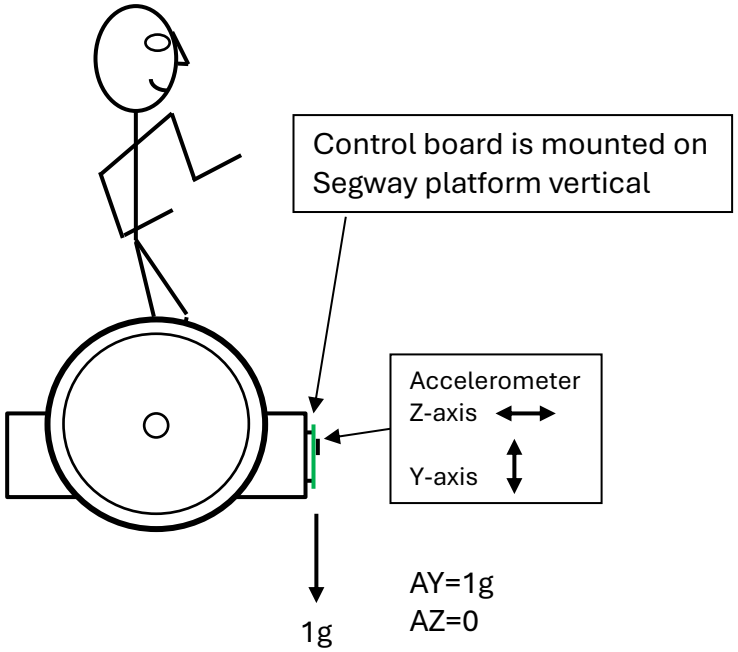
    (… we are subtracting due to orientation of the sensor mount on the platform)

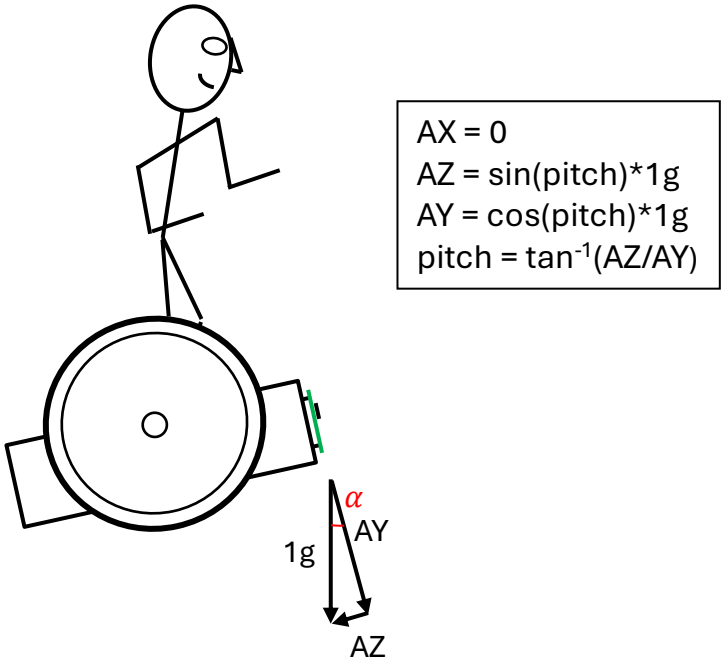
- When the Segway is maintaining level (not pitched forward or back), the pitch angular rate readings would be small -ve and small +ve numbers. This would on average sum to zero.

- However, just the act of soldering an inertial sensor to a board will affect the offset of its gyro readings. Even if it was perfectly factory compensated, it will not be once it is in your application.
- The small offsets, when integrated over time, will result in drift. We need to "fuse" the gyro readings with accelerometer readings (AZ) to correct for this drift.

Inertial Integrator – Sensor Fusion

- We can get pitch and roll from an accelerometer. The earth's gravity always provides 1g of acceleration straight down.





- Trigonometric Simplifications
  - For small angles (α) the $\tan^{-1}$(opposite/adjacent) ≈ opposite/adjacent
  - Taking it one step further for small angles: adjacent ≈ hypotenuse = unity
  - So… for us pitch is simply proportional to AZ.
  - This is only true for small angles, but the "Segway" platform better be at a small angle from horizontal or the rider is in serious trouble

Inertial Integrator – Sensor Fusion

- So why get pitch and roll from the gyro? We can just use AY and AZ
- Accelerometers are noisy, and Gyros are subject to long term drift.
  - Accelerometer readings are noisy (vibrations and linear acceleration)
  - Gyro readings are amazingly quiet (even in presence of vibration) but since we are integrating, any remaining offset error will result in a long term drift.
- Sensor fusion is an attempt to take the best from each to create a low noise, no drift version of pitch.
  - The fusion data is dominated by the integrated gyro readings, thus it stays low noise.
  - However, pitch is also calculated via accel readings, and the long term average of the integrated gyro readings is "leaked" toward the accelerometer results.
    (In electrical engineering terms the accelerometer gets to establish the DC operating point, but the gyro readings determine the transient response.)
  - **AZ[15:0]** will be used to calculate the pitch as seen by the accelerometer (**ptch_acc**).
    If **ptch_acc>ptch** then **ptch_int** will have a constant added to it.
    If **ptch_acc<ptch** then **ptch_int** will have a constant subtracted from it.

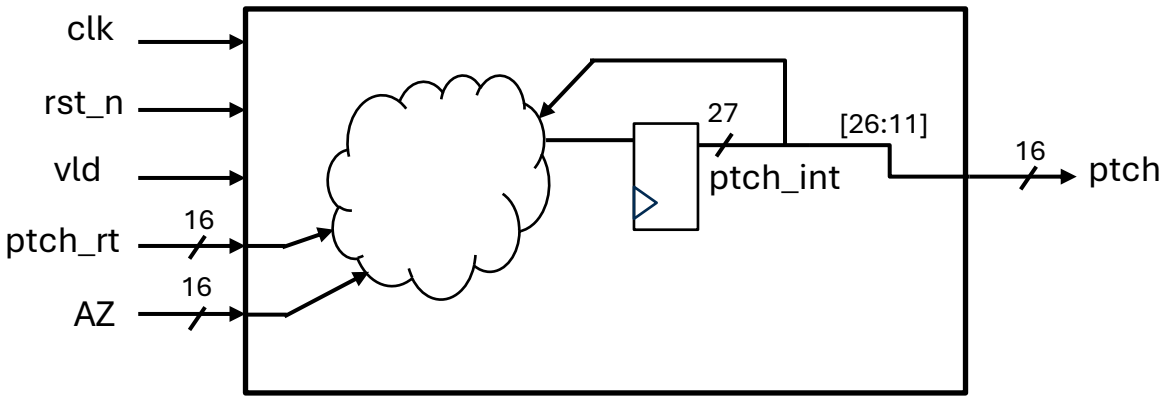
Inertial Integrator

- You will have the following interface for **inertial_integrator.sv**

Signal	Dir	Description
clk, rst_n	in	clock & reset
vld	in	High for a single clock cycle when new inertial readings are valid.
ptch_rt[15:0]	in	16-bit signed raw pitch rate from inertial sensor
AZ [15:0]	in	Will be used for sensor fusion (acceleration in Z direction)
ptch[15:0]	out	Fully compensated and "fused" 16-bit signed pitch.

- It should have an internal register:

Register	Purpose
ptch_int[26:0]	Pitch integrating accumulator. This is the register **ptch_rt** is summed into



Inertial Integrator

- The raw pitch rate readings (**ptch_rt**) has some offset due to how it's soldered on the platform. We will compute based on a compensated pitch rate:

  ```
  ptch_rt_comp = ptch_rt – PTCH_RT_OFFSET
  ```

  PTCH_RT_OFFSET = 16'h0050 (measured after soldering on Segway)

- On every **vld** pulse, we integrate (accumulate) the **ptch_rt_comp** signal:

  ```
  ptch_int = ptch_int – ptch_rt_comp   // without sensor fusion
  ```

  *(we integrate the **negative** of **ptch_rt_comp** due to orientation of sensor mount)*

- Bits [26:11] of **ptch_int** form **ptch[15:0]** (i.e., divided by 2^{11}). This scaling factor was derived by trial and error with the actual Segway.

- To perform sensor fusion with the accelerometer, we sum a fusion correction term (**fusion_ptch_offset**) into **ptch_int:**

  ```
  ptch_int = ptch_int – ptch_rt_comp + fusion_ptch_offset
  ```

- **fusion_ptch_offset** is computed based on the accelerometer AZ value. This also has an offset due to soldering…

  ```
  AZ_comp = AZ – AZ_OFFSET
  ```

  AZ_OFFSET = 16'h00A0 (measured after soldering on Segway)

Inertial Integrator

- With our trigonometric simplifications, the pitch according to the accelerometer is proportional to AZ:

```
ptch_acc_product = AZ_comp * $signed(327); // 327 is a fudge factor
ptch_acc = {{3{ptch_acc_product[25]}},ptch_acc_product[25:13]};
```

- We compute the fusion correction term based on whether **ptch_acc** is greater than our integrated **ptch** value:

```
if (ptch_acc > ptch)
  fusion_ptch_offset = 1024
else
  fusion_ptch_offset = -1024
```

- Note that we are doing signed computation. Make sure you declare signals as signed or are doing proper sign extension!

Testing Inertial Integrator

- Create a testbench that performs the following: (try to replicate our waveform)
- After reset has deasserted, apply **ptch_rt** = (16'h1000 + PTCH_RT_OFFSET) and **AZ** = 16'h0000.
  Ensure **vld** is held high.
  Maintain this stimulus for 500 clocks.
  The **ptch** output should be trending more and more negative since we integrate the negative of **ptch_rt**.
- Now zero out the pitch rate by applying a value of **ptch_rt** = PTCH_RT_OFFSET, and hold for 1000 clocks.
  Due to the fusion with the **AZ** reading of 0, you should see **ptch** reading trending slowly back towards 0.
- Now, set **ptch_rt** = (PTCH_RT_OFFSET – 16'h1000) for 500 clocks.
  The **ptch** output should trend steeply into positive territory.
- Again, zero out the pitch rate by setting **ptch_rt** = PTCH_RT_OFFSET, and hold for 1000 clocks.
  The **ptch** output should be trending slowly back toward zero.
- Finally adjust **AZ** = 16'h0800.
  When **ptch** gets down to about 100, it should level off as the ptch calculated from AZ should match that from fusion and the offset should start toggling back and forth from -1024 to +1024.

